

Лабораторна робота №1.

Створення базового проєкту з використанням FreeRTOS

Мета роботи. Встановити середовище розробки вбудованого програмного забезпечення та написати програму для управління блиманням світлодіодів використовуючи операційну систему реального часу FreeRTOS.

Хід виконання роботи.

1. Встановіть середовище розробки Visual Studio Code.
2. Встановіть офіційне розширення Raspberry Pi Pico для Visual Studio Code.
3. В домашньому каталозі створіть папку де ви будете зберігати ваші проєкти, наприклад: **rtos**.
4. Завантажте в каталог із вашими проєктами ядро операційної системи реального часу FreeRTOS використовуючи команду:

```
git clone https://github.com/raspberrypi/FreeRTOS-Kernel.git --recurse-submodules
```

5. Відкрийте Raspberry Pi Pico Project на панелі інструментів Visual Studio Code. Виберіть New C/C++ Project. У вікні яке відкриється:

- задайте ім'я проєкту, наприклад **lab01**;
- виберіть Board type: **Pico W**;
- у Location вкажіть каталог із вашими проєктами;
- в Stdio support виберіть **Console over USB**;
- натисніть кнопку **Create**, щоб створити проєкт.

Після чого потрібно трохи зачекати поки Visual Studio Code завантажить необхідні компоненти для компіляції вашого проєкту.

6. Відкрийте файл **CMakeLists.txt**, який знаходиться у каталозі вашого проєкту і відредагуйте його наступним чином:
 - знайдіть рядок із інструкцією - `set(PICO_BOARD pico_w CACHE STRING "Board type")`. Після цієї інструкції вставте `set(FREERTOS_KERNEL_PATH ../../FreeRTOS-Kernel)`. Замініть `../../` на повний шлях до каталогу з вашими проєктами, де знаходиться каталог FreeRTOS-Kernel;

- знайдіть рядок із інструкцією `include(pico_sdk_import.cmake)`. Після цієї інструкції вставте `include(${FREERTOS_KERNEL_PATH}/portable/ThirdParty/GCC/RP2040/FreeRTOS_Kernel_import.cmake)`;
- Знайдіть інструкцію - `target_link_libraries(lab01 pico_stdlib)` і додайте бібліотеки - FreeRTOS-Kernel і FreeRTOS-Kernel-Heap4.

```
1 target_link_libraries(lab-1
2     pico_stdlib
3     FreeRTOS-Kernel
4     FreeRTOS-Kernel-Heap4)
```

7. Завантажте в каталог вашого проєкту конфігураційний файл ядра операційної системи FreeRTOSConfig.h перейшовши за посиланням:

<https://github.com/mpavlyk/rtos/blob/main/lib/FreeRTOSConfig.h>

8. Відредагуйте файл з кодом вашого проєкту:

```
1 #include <stdio.h>
2 #include "pico/stdlib.h"
3 #include <FreeRTOS.h>
4 #include <task.h>
5
6 void blink_task(void *pvParameters) {
7     const uint LED_PIN = 10;
8     gpio_init(LED_PIN);
9     gpio_set_dir(LED_PIN, true);
10
11     while (true) {
12         gpio_put(LED_PIN, 1);
13         vTaskDelay(pdMS_TO_TICKS(500));
14         gpio_put(LED_PIN, 0);
15         vTaskDelay(pdMS_TO_TICKS(500));
16     }
17 }
18
19 int main()
20 {
21     stdio_init_all();
22
23     xTaskCreate(blink_task, "Blink", 256, NULL, 1, NULL);
24
25     vTaskStartScheduler();
26 }
```

27	while (true) {
28	printf("Hello, world!\n");
29	sleep_ms(1000);
30	}
31	}

9. Скомпілюйте проєкт та завантажте файл з розширенням .uf2 на мікроконтролер Raspberry Pi Pico. Для цього потрібно зажати кнопку BOOTSEL на платі з мікроконтролером і підключити плату до комп'ютера за допомогою кабелю USB. Мікроконтролер підключиться до комп'ютера як змінний диск. На цей змінний диск потрібно скопіювати файл з розширенням .uf2 з каталогу build вашого проєкту.
10. Напишіть програму, щоб одночасно блимав світлодіод на платі мікроконтролера і світлодіод на платі Pico-Sensor-Kit, а також виводилося повідомлення.